

MyTrip System Structure and Algorithms Report

MyTrip – Budget Road Trip Planner

CS 4398 – Software Engineering Project

Texas State University

Summer 2026 **Prepared**

by:

Mustafa Al Azzawi

Angelica Alvarado William
Parker

Table of Contents

1. Project Overview
2. System Architecture
3. Package Structure
4. Model Layer
5. Controller Layer
6. Service Layer
7. Repository and H2 Database
8. Frontend Structure
9. Fuel-Cost Algorithm
10. Total-Cost and Budget Algorithms
11. Drive-Time Estimation
12. CRUD Operations
13. Trip Ownership by Traveler Email
14. Validation and Exception Handling
15. Data Structures
16. External and Simulated Data
17. Known Limitations
18. Conclusion

1. Project Overview

MyTrip – Budget Road Trip Planner is a web-based application created to help travelers organize a road trip and understand its likely cost before leaving. The system addresses a common planning problem: route information, fuel cost, lodging, activities, and budget decisions are often handled in separate places. MyTrip brings these planning tasks into one academic prototype.

The target users are travelers who want a straightforward way to build and save a trip. A traveler signs in with a demonstration account, enters a starting location, an optional waypoint, a destination, route mileage, a target budget, and vehicle information. The application displays the route with Google Maps, estimates fuel cost and drive time, presents simulated lodging, fuel-station, and attraction choices, and summarizes the expected trip cost.

The main workflow is to plan a route, review route options, select relevant trip items, compare the calculated total with the budget, and save the trip under the traveler's email. A saved trip can later be reopened or deleted. The current integration version demonstrates the full saved-trip CRUD workflow required for the academic prototype.

2. System Architecture

MyTrip uses a layered Spring Boot architecture. Separating presentation, request handling, business logic, data access, and persistence makes the system easier to understand, test, and maintain.

Presentation layer: HTML, CSS, and JavaScript collect input and display route, selection, account, and summary information in the browser.

Controller layer: TripController receives HTTP or REST requests and returns application responses.

Service layer: Service layer: TripService coordinates backend validation, saved-trip business rules, trip lookup, create, update, and delete operations, and repository access. Calculated Trip values such as fuel cost, total cost, budget difference, and under-budget status are derived from the Trip model, while drive-time display and several interactive planning calculations are performed in frontend JavaScript.

Repository layer: TripRepository provides Spring Data JPA access to stored Trip records.

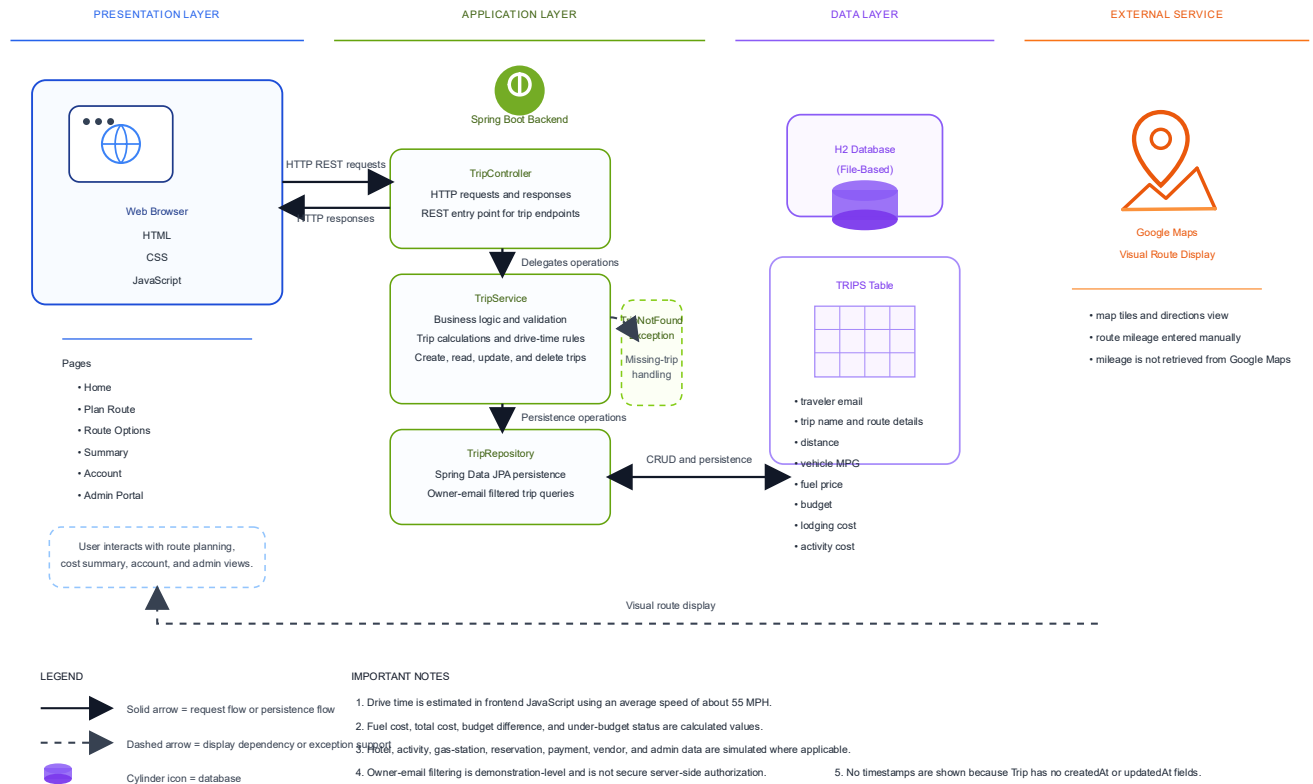
Data layer: A file-based H2 database stores saved trip information locally.

External route display: Google Maps displays the route and gives the traveler a visual mileage reference.

The normal backend request flow is:

Web Browser → TripController → TripService → TripRepository → H2 Database

Responses travel back through the backend and are returned to the browser as application data. TripController does not communicate directly with the database; repository access is coordinated through TripService.



3. Package Structure

The backend is organized under the base package com.mytrip. An appropriate Spring Boot organization separates classes by responsibility while keeping related behavior together.

The application entry point contains MytripApplication and starts the Spring Boot application.

The controller area contains request-handling logic such as TripController.

The service area contains business logic such as TripService.

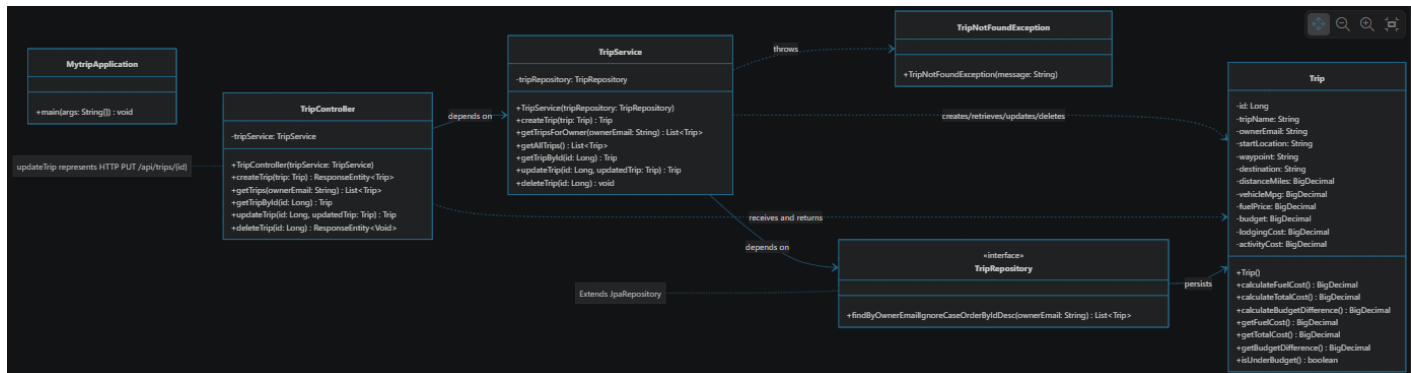
The repository area contains data-access interfaces such as TripRepository.

The model or entity area contains the Trip entity.

The exception-handling area contains application-specific errors such as TripNotFoundException.

Frontend static resources contain the HTML pages, CSS styles, JavaScript behavior, and related assets.

This organization supports clear responsibilities without requiring the frontend, controller, service, and repository code to be combined in the same package.



4. Model Layer

The Trip entity represents a saved road-trip plan. It carries the information needed to identify the trip, associate the record with its demonstration owner, store planning inputs, and derive cost results.

Stored Trip fields include:

- trip ID;
- trip name;
- traveler email;
- starting location;
- optional waypoint;
- destination;
- route distance;
- vehicle MPG;
- fuel price per gallon;
- target budget;
- lodging cost; and
- activity cost.

The trip ID distinguishes one saved record from another. Traveler email is used to filter the account view for the academic demonstration. Route distance, vehicle MPG, fuel price, budget, lodging cost, and activity cost provide the inputs used by the planning calculations.

Fuel cost, total estimated cost, budget difference, and under-budget status are calculated or transient values derived from the stored inputs. They are not separate timestamp or record-history fields. The submitted Trip entity does not contain createdAt or updatedAt timestamps.

5. Controller Layer

TripController is responsible for receiving HTTP or REST requests from the frontend and communicating with TripService. It acts as the boundary between browser requests and backend application logic.

Controller operations support creating or saving trips, retrieving all trips when appropriate, retrieving one trip by ID, retrieving trips filtered by traveler email, updating an existing trip through HTTP PUT, and deleting a saved trip.

TripController delegates backend business rules, validation, lookup, update, and persistence behavior to TripService rather than accessing H2 directly. The controller receives and returns Trip objects through the REST API.

6. Service Layer

TripService contains the central backend business logic for saved Trip records. It coordinates data received from TripController, validates important saved-trip values, looks up Trip records, copies updated values into existing records, and uses TripRepository for persistence.

Its implemented responsibilities include:

- creating and saving Trip records;
- retrieving all Trip records when requested;
- retrieving records filtered by traveler email;
- finding a Trip by ID;
- updating an existing Trip;
- deleting a Trip;
- preserving the existing Trip ID during an update; and
- throwing TripNotFoundException when a requested record does not exist.

TripService does not directly generate the visual route and does not retrieve live mileage or live drive time from Google Maps. Interactive drive-time estimation and several planning-display calculations occur in frontend JavaScript. Calculated cost properties are also derived from Trip values rather than being stored as separate database timestamps or history records.

Keeping persistence operations in the service layer preserves the Controller → Service → Repository separation and prevents TripController from communicating with H2 directly.

7. Repository and H2 Database

TripRepository is the Spring Data JPA data-access interface for Trip records. Spring Data JPA supplies standard create, read, update, and delete behavior and supports lookup of trips associated with a traveler email. TripRepository also defines `findByOwnerIgnoreCaseOrderByIdDesc(String ownerEmail)` to retrieve saved trips for the demonstration account view.

MyTrip uses file-based local H2 persistence with the following connection location:

`jdbc:h2:file:./data/mytrip`

Because the database is file-based, saved trips remain after a browser page refresh and can remain after an application restart. However, each local installation uses its own database file. A trip saved on one team member's computer is not automatically shared with another computer.

The H2 console can be accessed in the local development environment. Stored trip records can be inspected with:

`SELECT * FROM TRIPS;`

8. Frontend Structure

The presentation layer uses HTML for page structure, CSS for layout and visual styling, and JavaScript for interaction, validation, calculation display, selections, and communication with the backend.

Home: introduces MyTrip and provides access to the main planning workflow.

Plan Route: collects the starting location, optional waypoint, destination, budget, mileage, vehicle, and traveler information.

Route Options: displays the Google Maps route, manually entered mileage, estimated drive time, and available planning options.

Summary: shows fuel, lodging, activity, total, budget, and group-splitting results.

Account: shows trips filtered by the active demonstration email and supports opening, editing, updating, and deleting saved trips.

Admin Portal: provides demonstration-level administrative access for the academic prototype.

The browser displays external Google Maps route information but uses mock lodging, attraction, and fuel-station listings for the prototype. JavaScript sends relevant trip requests to the Spring Boot backend and displays the returned application data.

The frontend stores the ID of the trip currently being edited. Saving a new trip sends an HTTP POST request, while submitting changes to an existing trip sends an HTTP PUT request to `/api/trips/{id}`. After a successful update, the saved-trip list is refreshed with the latest values.

9. Fuel-Cost Algorithm

The fuel-cost calculation uses route distance, vehicle efficiency, and fuel price. Route distance and MPG must both be greater than zero.

$$\text{Gallons Required} = \text{Route Distance} \div \text{Vehicle MPG}$$

$$\text{Estimated Fuel Cost} = \text{Gallons Required} \times \text{Fuel Price Per Gallon}$$

For the tested example, the route distance is 81.9 miles, the vehicle efficiency is 28 MPG, and the fuel price is approximately \$3.38 per gallon.

$$81.9 \div 28 = 2.925 \text{ gallons}$$

$$2.925 \times \$3.38 \approx \$9.89$$

The estimated fuel cost is therefore approximately \$9.89. Minor display differences can occur if the demonstration fuel price changes or if intermediate values are rounded differently.

10. Total-Cost and Budget Algorithms

Selected activity prices are added together before the final estimate is calculated.

$$\text{Activity Cost} = \sum \text{Selected Activity Price}_i, \text{ for } i = 1 \text{ to } n$$

$$\text{Total Estimated Cost} = \text{Fuel Cost} + \text{Lodging Cost} + \text{Activity Cost}$$

$$\text{Budget Difference} = \text{Budget} - \text{Total Estimated Cost}$$

If the budget difference is zero or positive, the trip is within budget. If the value is negative, the trip is over budget. The summary also supports group cost splitting by dividing the total estimated cost among the number of travelers. The traveler count must be valid before a per-person amount can be calculated.

11. Drive-Time Estimation

MyTrip estimates drive time in frontend JavaScript from the manually entered route mileage and an assumed average driving speed of approximately 55 MPH. It does not retrieve live travel time directly from Google Maps.

$$\text{Estimated Drive Time} = \text{Route Distance} \div \text{Average Speed}$$

Using the tested values of 81.9 miles and an average speed of 55 MPH:

$$81.9 \div 55 \approx 1.489 \text{ hours} \approx 1 \text{ hour } 29 \text{ minutes}$$

The estimated drive time is displayed by MyTrip under the map on the Route Options page. It is a planning estimate and does not account for current traffic, construction, rest stops, or changing road conditions.

12. CRUD Operations

The current backend and frontend support the complete saved-trip CRUD workflow:

Create: A new Trip is sent through HTTP POST and saved through TripService and TripRepository.

Read: The application can retrieve all trips when appropriate, retrieve one trip by ID, and retrieve trips filtered by traveler email.

Update: Selecting Edit loads an existing saved Trip into the planning form and records its ID.

Submitting the revised information sends an HTTP PUT request to `/api/trips/{id}`. TripService updates the existing record and preserves its ID.

Delete: Selecting Delete sends the saved Trip ID to the backend. TripService verifies that the record exists and then deletes it through TripRepository.

Reopening a saved trip restores its information for review. Editing and updating are implemented in the current integration version and should be treated as completed prototype functionality.

13. Trip Ownership by Traveler Email

Each saved Trip is associated with a traveler email address. The account view requests and displays records filtered by that email. This separates the saved trip lists shown for the demonstration accounts.

During acceptance testing, a trip saved under `traveler@example.com` did not appear in the account list when `traveler2@example.com` was active. This confirms the intended frontend and repository filtering behavior for the academic prototype.

This email-based filtering is not secure authentication or server-side authorization. A production system would require verified identities, authenticated sessions, and backend access-control checks before allowing ID-based read, update, or delete operations.

14. Validation and Exception Handling

Validation protects calculations and persistence from incomplete or invalid input. Important checks include route mileage greater than zero, budget greater than zero, valid MPG values, a required starting location, a required destination, and a signed-in traveler before saving.

Observed validation messages include:

“Enter a valid route distance greater than zero.”

“Error: Invalid target budget.”

Guest users are prevented from saving trips. `TripNotFoundException` is the custom exception used when a requested trip cannot be found. The application can translate validation failures and missing-trip errors into responses that the frontend can display to the traveler.

15. Data Structures

The primary data structure is the Trip object, which groups the fields and calculated values for one saved plan. Collections are used conceptually wherever the application must display or process more than one item. lists of saved trips; lists of lodging options; lists of attractions; lists of fuel stations; collections of selected options;

numeric values for distances, prices, costs, budgets, and traveler counts; and a Boolean value for under-budget status.

Lists are appropriate because the number of saved trips or selectable options can vary. They also allow the frontend to repeat a consistent display component for each result. Numeric and Boolean values support direct calculations and clear budget-status decisions.

16. External and Simulated Data

External Data

Google Maps provides the visual route display and map information. The displayed route mileage serves as a reference, but the traveler manually enters that mileage into MyTrip. The application does not automatically retrieve route distance from Google Maps.

Simulated or Mock Data

lodging listings; attraction listings; fuel-station listings; booking actions; reservation confirmations; payment behavior; and changing demonstration gas-price values.

These simulated elements allow the team to demonstrate the complete planning experience without connecting to commercial reservation or payment services. MyTrip does not perform real hotel reservations, real activity bookings, or real payments.

17. Known Limitations

MyTrip is an academic prototype, and its limitations should be understood when reviewing the current implementation.

Manual route mileage:

Route distance is not retrieved automatically. The traveler enters the mileage shown by the route reference.

Estimated drive time:

Drive time uses an assumed average speed rather than live Google Maps traffic or travel-time information.

Mock travel content:

Hotel, activity, and fuel-station information is simulated rather than retrieved from production providers.

Simulated transactions:

Reservations, confirmations, vendor actions, and payments are demonstrations only.

Demonstration login and filtering:

Login behavior and traveler-email filtering are suitable for demonstrating account-specific views but do not provide secure server-side authentication or authorization.

Local persistence:

Each installation uses its own local H2 database file, so data is not shared automatically across computers.

Manual document maintenance:

The UML and architecture diagrams are static project artifacts and must be updated manually when the implementation changes.

Minor interface issues:

Small formatting or interface inconsistencies may remain in the prototype.

These limitations are acceptable for the current course prototype because the system demonstrates layered architecture, validation, persistence, CRUD behavior, cost calculations, route visualization, and the primary traveler workflow. A future production version could add secure authentication, server-hosted shared persistence, automatic route-distance and traffic data, real provider integrations, transaction security, and deployment monitoring.

18. Conclusion

MyTrip combines an HTML, CSS, and JavaScript browser interface with a Spring Boot backend written for Java 21. TripController receives REST requests, TripService coordinates backend validation and saved-trip business logic, TripRepository provides Spring Data JPA persistence, and H2 stores saved Trip records locally.

Google Maps supplies the visual route display, while route mileage is entered manually. The application derives fuel cost, total estimated cost, budget difference, under-budget status, drive-time estimates, and group cost splitting from the traveler's planning inputs.

The system demonstrates the primary road-trip planning workflow: a traveler can use a demonstration login, create a route plan, enter route mileage, review simulated travel options, compare the estimate with a budget, save the trip, reopen it, edit it, update it, and delete it. Traveler-email filtering separates the account lists displayed for the demonstration users but is not secure server-side authorization.

The current version meets the main goals of an academic software engineering prototype. Its remaining limitations are production authentication, real external booking and payment services, automatic mileage and traffic retrieval, shared deployment data, and production-grade transaction security.